Chapter 05 · SQL Databases

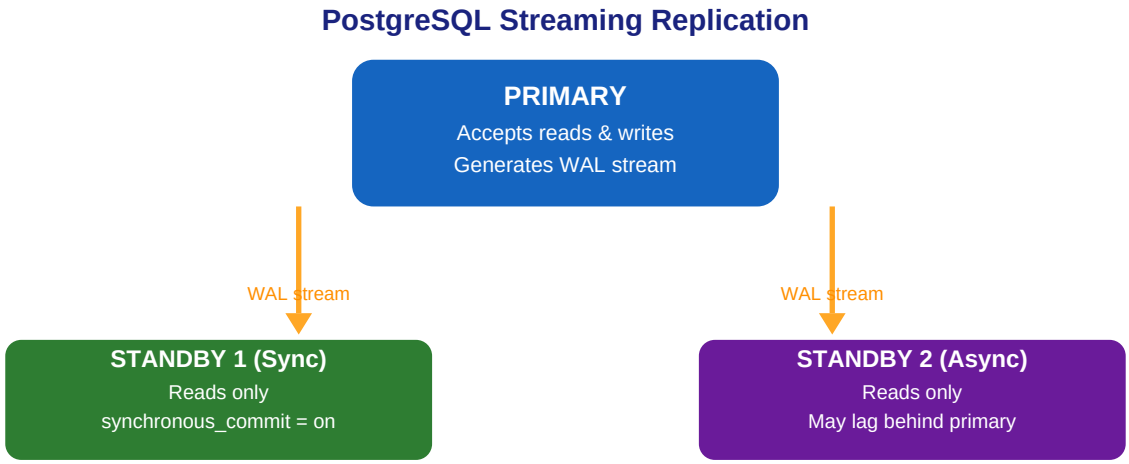
Sub-Chapter 09 — Replication & High Availability

Scope	Streaming replication, Patroni HA, read scaling, PgBouncer
Database	PostgreSQL with Patroni + etcd
Level	Advanced
Outcome	Build zero-downtime, read-scalable Spring DB clusters

1 Why Replication & HA?

Survive failures and scale reads without downtime

A single PostgreSQL server is a **single point of failure**. If it crashes, your application is down. **Replication** creates one or more copies of your data on other servers. **High Availability (HA)** means the system automatically promotes a replica to primary when the primary fails — with minimal downtime and no data loss.



Sync: primary waits for standby ACK before commit — zero data loss, slight latency
Async: primary does not wait — higher throughput, risk of losing last few transactions on failover

Zero data loss — Synchronous replication: standby must confirm write before primary commits.

Low latency — Asynchronous replication: primary commits immediately, replica catches up.

Read scaling — Route SELECT queries to replicas — primary handles only writes.

Automatic failover — Tools like Patroni + etcd elect a new primary within seconds of a crash.

Geo-redundancy — Hot standby in another data centre / availability zone survives a site outage.

2 PostgreSQL Streaming Replication

WAL-based physical replication — the foundation of all HA setups

PostgreSQL writes every change to the **Write-Ahead Log (WAL)** before touching data files. Streaming replication ships those WAL records to standby servers in real time. The standby replays them, staying nearly in sync.

2.1 Setting Up Streaming Replication (Quick Overview)

```
-- On PRIMARY: postgresql.conf wal_level = replica -- must be replica or logical max_wal_senders = 10 -- allow up to 10 standbys wal_keep_size = 1024 -- MB: keep WAL for slow standbys hot_standby = on -- standby can serve SELECT -- On PRIMARY: pg_hba.conf — allow replication connections host replication replicator 10.0.0.0/24 scram-sha-256 -- Create replication user CREATE ROLE replicator REPLICATION LOGIN PASSWORD 'secret'; -- On STANDBY: take base backup from primary pg_basebackup -h primary-host -U replicator -D /var/lib/postgresql/data -P -R # -R creates standby.signal and fills primary_conninfo automatically -- Start standby — it will stream WAL from primary automatically
```

2.2 Monitoring Replication Lag

```
-- On PRIMARY: see all connected standbys and their lag SELECT application_name, state, sent_lsn, write_lsn, flush_lsn, replay_lsn, (sent_lsn - replay_lsn) AS replication_lag_bytes, sync_state -- async / sync / quorum FROM pg_stat_replication; -- On STANDBY: see how far behind it is SELECT
```

```
now() - pg_last_xact_replay_timestamp() AS replication_delay, pg_is_in_recovery() AS is_standby;
```

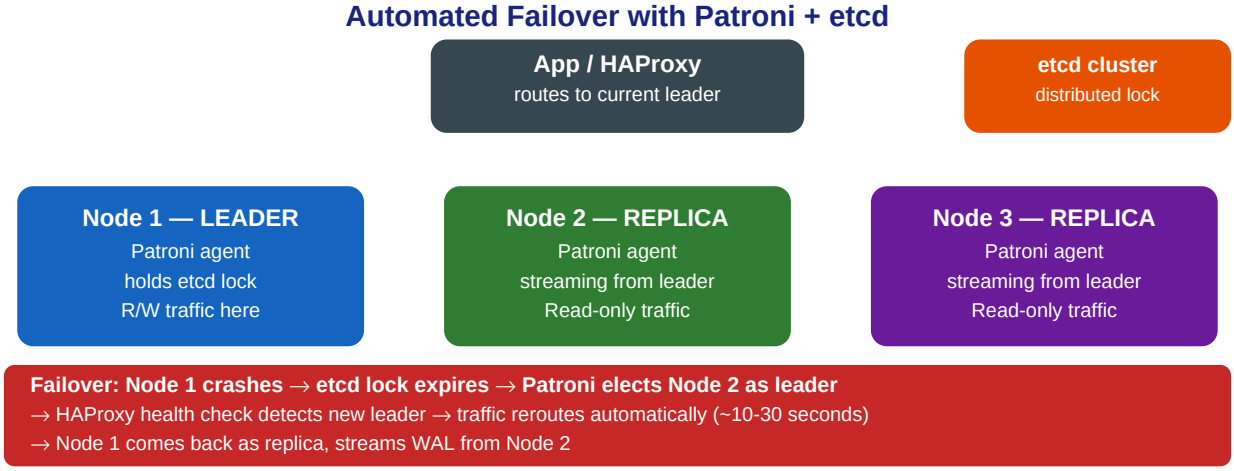
2.3 Synchronous vs Asynchronous

```
-- postgresql.conf on PRIMARY: -- Async (default): no wait for standby, best throughput
synchronous_commit = off -- Sync: wait for ONE standby to flush before commit returns
synchronous_standby_names = 'standby1' synchronous_commit = on -- Quorum: wait for ANY 2 out of 3
standbys (PostgreSQL 10+) synchronous_standby_names = 'ANY 2 (standby1, standby2, standby3)'
```

With **synchronous** replication you get zero data loss — if the primary crashes after a commit, the standby has that commit. Trade-off: ~1-5 ms extra latency per write (the network round-trip to standby).

3 Automated Failover with Patroni

Industry-standard HA solution for PostgreSQL



Patroni is a Python daemon that runs alongside each PostgreSQL node. It uses a distributed consensus store (**etcd**, Consul, or ZooKeeper) to elect a leader and trigger automatic failover when the leader becomes unavailable.

3.1 How Failover Works (Step by Step)

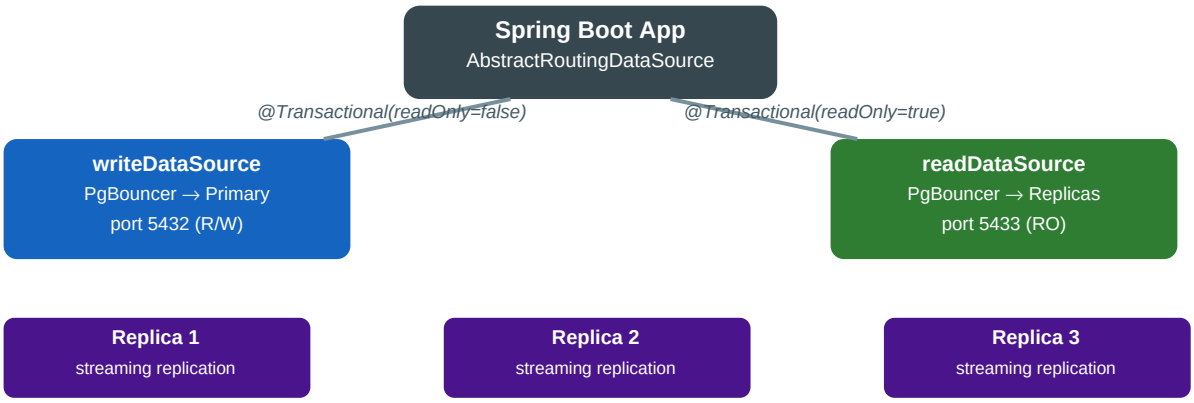
1. Primary node crashes or becomes unreachable.
2. Patroni agents on replicas detect the leader's etcd lease has expired (TTL ~10s).
3. Replicas participate in election — the one with the least replication lag wins.
4. Winner promotes itself: pg_ctl promote — it becomes the new primary.
5. Other replicas reconfigure to stream from the new primary.
6. HAProxy / PgBouncer health check detects new leader via Patroni REST API.
7. Application connections are routed to the new primary automatically.

3.2 Patroni Configuration Snippet (patroni.yml)

```
scope: postgres-cluster name: node1 restapi: listen: 0.0.0.0:8008 connect_address: 10.0.0.1:8008
etcd3: hosts: 10.0.0.10:2379,10.0.0.11:2379,10.0.0.12:2379 bootstrap: dcs: ttl: 30 loop_wait: 10
retry_timeout: 10 maximum_lag_on_failover: 1048576 # 1 MB max lag for promotion synchronous_mode:
false postgresql: parameters: wal_level: replica hot_standby: "on" max_wal_senders: 10 postgresql:
listen: 0.0.0.0:5432 connect_address: 10.0.0.1:5432 data_dir: /var/lib/postgresql/data
authentication: replication: username: replicator password: secret
```

4 Read Scaling with Replicas

Route reads to standbys — multiply your read throughput



Most applications read far more than they write. Adding read replicas and routing SELECT queries to them multiplies read throughput linearly. Spring's **AbstractRoutingDataSource** makes this transparent to repositories.

4.1 AbstractRoutingDataSource — Read/Write Split

```
// 1. Context holder to track current data source public class DataSourceContextHolder { private static final ThreadLocal ctx = new ThreadLocal<>(); public static void setRead() { ctx.set("read"); } public static void setWrite() { ctx.set("write"); } public static String get() { return ctx.get(); } public static void clear() { ctx.remove(); } } // 2. Routing DataSource public class RoutingDataSource extends AbstractRoutingDataSource { @Override protected Object determineCurrentLookupKey() { return DataSourceContextHolder.get(); } } // 3. DataSource configuration @Configuration public class DataSourceConfig { @Bean @ConfigurationProperties("spring.datasource.write") public DataSource writeDataSource() { return DataSourceBuilder.create().build(); } @Bean @ConfigurationProperties("spring.datasource.read") public DataSource readDataSource() { return DataSourceBuilder.create().build(); } @Bean @Primary public DataSource routingDataSource( @Qualifier("writeDataSource") DataSource write, @Qualifier("readDataSource") DataSource read) { RoutingDataSource ds = new RoutingDataSource(); ds.setDefaultTargetDataSource(write); ds.setTargetDataSources(Map.of("write", write, "read", read)); ds.afterPropertiesSet(); return ds; } }
```

4.2 AOP Aspect to Auto-Route by @Transactional(readOnly)

```
@Aspect @Component @Order(0) // must run BEFORE @Transactional public class ReadWriteRoutingAspect { @Around("@annotation(tx)") public Object route(ProceedingJoinPoint pjp, Transactional tx) throws Throwable { try { if (tx.readOnly()) { DataSourceContextHolder.setRead(); } else { DataSourceContextHolder.setWrite(); } return pjp.proceed(); } finally { DataSourceContextHolder.clear(); } } } // Usage: nothing changes in your service! @Service public class ProductService { @Transactional(readOnly = true) // --> routed to read replica public List findAll() { return repo.findAll(); } @Transactional // --> routed to primary public Product save(Product p) { return repo.save(p); } }
```

5 Connection Pooling with PgBouncer

Multiplexing connections between app and PostgreSQL

PostgreSQL creates a OS process per connection (~5-10 MB RAM each). A Spring app with 20 pods × 10 pool connections = 200 PostgreSQL processes. **PgBouncer** sits between your app and PostgreSQL, multiplexing many app connections onto a small number of server connections.

PgBouncer Modes

Session pooling — One server connection per client session. Least efficient — basically passthrough.

Transaction pooling — Server connection returned to pool after each transaction. Recommended — works with Spring's connection pool.

Statement pooling — Server connection released after each statement. Cannot use prepared statements or multi-statement transactions.

```
# pgbouncer.ini (transaction pooling) [databases] mydb = host=primary-pg port=5432 dbname=mydb
[pgbouncer] pool_mode = transaction listen_addr = 0.0.0.0 listen_port = 5432 max_client_conn =
1000 # app-facing connections default_pool_size = 25 # server-side connections per user/db
server_idle_timeout = 600 log_connections = 0
```

application.yml with PgBouncer

```
spring: datasource: write: url: jdbc:postgresql://pgbouncer-write:5432/mydb hikari:
maximum-pool-size: 10 # HikariCP pool to PgBouncer minimum-idle: 2 connection-timeout: 3000 #
IMPORTANT: disable preparedStatement cache with PgBouncer transaction mode data-source-properties:
prepareThreshold: 0 # disables server-side prepared statements read: url:
jdbc:postgresql://pgbouncer-read:5433/mydb hikari: maximum-pool-size: 20 data-source-properties:
prepareThreshold: 0
```

6 Monitoring Replication Health

Key metrics and Spring Actuator integration

6.1 Key PostgreSQL Views

```
-- Replication lag (run on primary) SELECT client_addr, state, ROUND((sent_lsn - replay_lsn) /
1024.0 / 1024.0, 2) AS lag_mb, sync_state FROM pg_stat_replication ORDER BY lag_mb DESC; --
Identify if current node is standby or primary SELECT pg_is_in_recovery() AS is_replica; -- Last
replay time on standby SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag,
pg_last_xact_replay_timestamp() AS last_replayed_at; -- Slot lag (dangerous if replica disconnects
— WAL accumulates) SELECT slot_name, pg_size_pretty( (pg_current_wal_lsn() - restart_lsn)) AS
retained_wal FROM pg_replication_slots;
```

6.2 Spring Boot Actuator + Custom Health Indicator

```
@Component public class ReplicationLagHealthIndicator implements HealthIndicator { private final
JdbcTemplate jdbc; private static final long MAX_LAG_SECONDS = 30; @Override public Health
health() { try { // Query to run on the primary Long lagBytes = jdbc.queryForObject( "SELECT
COALESCE(MAX(sent_lsn - replay_lsn), 0) FROM pg_stat_replication", Long.class); if (lagBytes !=
null && lagBytes > 50 * 1024 * 1024) { // 50 MB return Health.down()
.withDetail("replication_lag_bytes", lagBytes) .withDetail("message", "Replica lag exceeds 50 MB")
.build(); } return Health.up() .withDetail("replication_lag_bytes", lagBytes) .build(); } catch
(Exception e) { return Health.down(e).build(); } } } // application.yml — expose health endpoint
management: endpoints: web: exposure: include: health,metrics,info endpoint: health: show-details:
always
```

7 Quick-Reference Cheatsheet

Commands and config for replication and HA

Topic	Command / Config	Notes
Enable WAL streaming	wal_level = replica in postgresql.conf	Restart required

Base backup for standby	pg_basebackup -h primary -U replicator -D /data -R	-R writes recovery config
Check replication status	SELECT * FROM pg_stat_replication	Run on primary
Check standby lag	SELECT now() - pg_last_xact_replay_timestamp()	Run on standby
Sync replication	synchronous_standby_names = 'standby1'	In postgresql.conf
Manual failover (Patroni)	patronictl -c patroni.yml failover cluster	Graceful switchover
Promote standby manually	pg_ctl promote -D /data OR SELECT pg_promote()	Makes standby a primary
Spring read routing	AbstractRoutingDataSource + AOP @Transactional aspect	
PgBouncer pool mode	pool_mode = transaction in pgbouncer.ini	Best for Spring apps
Disable PS cache	prepareThreshold=0 in datasource props	Required with PgBouncer tx mode